In TypeScript, `export` makes code available from one file, and `import` brings that code into another file.

Think of it as:

> **File A → export → File B → import**

1. Named export

math.ts

```typescript
export const add = (a: number, b: number): number => {
    return a + b;
};

export const subtract = (a: number, b: number): number => {
    return a - b;
};
```

Now another file can import them.

app.ts

```typescript
import { add, subtract } from "./math";

console.log(add(10, 5));
console.log(subtract(10, 5));
```

The `{ }` means you are importing **named exports**.

---

2. Export a function

You can directly export a function:

```typescript
export function greet(name: string): string {
    return `Hello, ${name}`;
}
```

Import it:

TypeScript

```
import { greet } from "./greetings";

console.log(greet("Bala"));
```

---

3. Default export

A file can have **one default export**.

calculator.ts

TypeScript

```
const multiply = (a: number, b: number): number => {
    return a * b;
};

export default multiply;
```

Import it:

TypeScript

```
import multiply from "./calculator";

console.log(multiply(5, 4));
```

Notice that there are **no** `{ }` around `multiply`.

---

4. Named vs default export

Named Export	Default Export
`export const add = ...`	`export default add`
Import using `{ }`	Import without `{ }`
Can have multiple	One default per file
`import { add } from "./math"`	`import add from "./math"`

5. Import everything

You can import all named exports:

```typescript
import * as MathUtils from "./math";

console.log(MathUtils.add(10, 5));
console.log(MathUtils.subtract(10, 5));
```

Here, `MathUtils` becomes an object containing the exported functions.

6. Very common Playwright/automation example

This is particularly useful when organizing test automation code.

loginPage.ts

```typescript
export class LoginPage {

    login(username: string, password: string): void {
        console.log(`Logging in as ${username}`);
    }
}
```

loginTest.ts

```typescript
import { LoginPage } from "./loginPage";

const loginPage = new LoginPage();

loginPage.login("admin", "password123");
```

So in test automation, you might have:

```
pages/
   LoginPage.ts
   HomePage.ts

tests/
   LoginTest.ts
```

and use `export` / `import` to share your Page Objects, utility functions, test data, interfaces, etc.

Simple rule to remember:

```
export = "Make this available to other files"

import = "Bring something from another file"
```

For your TypeScript-for-test-automation learning, **imports/exports are important before you move into Playwright Page Objects**, because Playwright projects use them heavily.